

MLP Notation: Naming Weights, Biases and Outputs

Contents

1. Overview	2
2. The setup: layers and data	3
2.1 Numbering the layers	3
2.2 Naming the inputs	3
3. Counting trainable parameters	4
4. Naming biases and outputs	6
4.1 Biases	6
4.2 Outputs	6
5. Naming weights	7
6. Summary	9
7. Sources	9
8. Key terms	9

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Leads to:** Multi-layer perceptron (MLP) ([Note 1009](#)); Forward propagation ([Note 1010](#)).

1. Overview

Key point: Before training a multi-layer perceptron, we count its trainable parameters and give every weight, bias and output a name that says exactly where it sits.

A multi-layer perceptron (MLP) is many perceptrons organised in layers (see the [types of neural networks](#) [Note](#)). Even a small one has dozens of weights and biases. Training it with backpropagation, in later Notes, means talking about each of them one at a time, so each needs an unambiguous name.

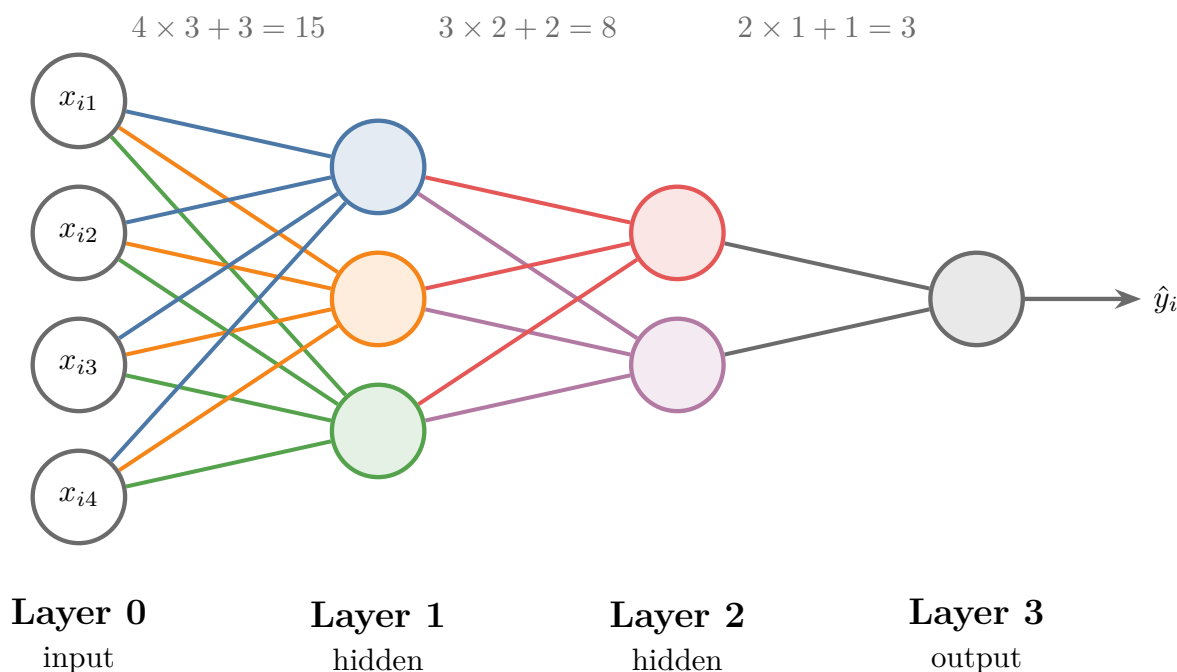


Figure 1: The 4-3-2-1 network used in this Note: layers 0 to 3, with the weights coming into each node drawn in that node's colour

This Note does two things, both on the network in Figure 1:

1. Count its **trainable parameters**: the weights and biases that training must find.
2. Fix a standard **notation** for biases, outputs and weights.

2. The setup: layers and data

Key point: Layers are numbered from 0 (the input layer) to the output layer; observation i of the data enters the input layer as $x_{i1}, x_{i2}, \dots$

2.1 Numbering the layers

Key point: Input layer 0, hidden layers 1 and 2, output layer 3.

The network in Figure 1 has four layers:

- **Layer 0**, the input layer: 4 nodes, one per **feature** (an input variable, one column of the data table). These nodes do no calculation; they only pass the values on.
- **Layer 1**, the first hidden layer: 3 perceptrons.
- **Layer 2**, the second hidden layer: 2 perceptrons.
- **Layer 3**, the output layer: 1 perceptron, whose output is the prediction $\hat{y}_i$.

We describe such a network by its layer sizes: a **4-3-2-1 network**. Input, hidden and output layers are defined in the [what is deep learning Note](#).

2.2 Naming the inputs

Key point: x_{ij} is the value of feature j for observation i : row i , column j of the data table.

The data has m **observations** (records, one row of the table per student) and $n = 4$ features, plus the **target** (the output we predict, here Placed):

Row	CGPA	IQ	10th marks	12th marks	Placed
1	7.2	72	69	81	1
...	...	...	...	...	...
i	x_{i1}	x_{i2}	x_{i3}	x_{i4}	y_i

The network takes the data one observation at a time. For observation i , the four values $x_{i1}, x_{i2}, x_{i3}, x_{i4}$ enter the four input nodes, and the network produces $\hat{y}_i$. For observation 1 above, 7.2 enters the first input node, 72 the second, 69 the third and 81 the fourth.

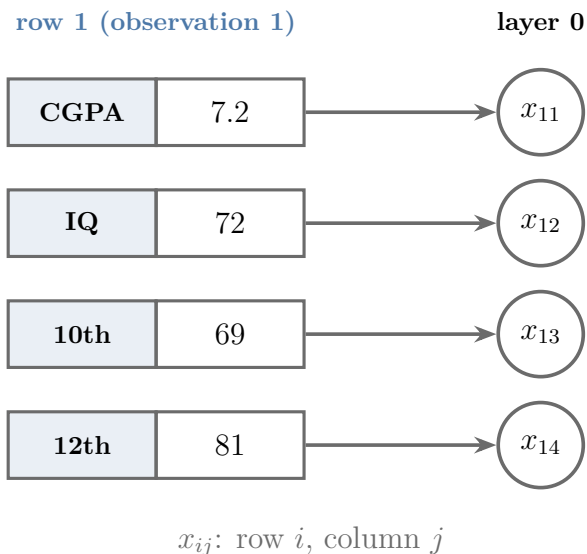


Figure 2: Observation 1 enters the network: each feature value of row 1 goes to its own input node, x_{11} to x_{14}

Figure 2 shows the rule behind the names: the first index of x_{ij} is the row, the second is the column.

3. Counting trainable parameters

Key point: Between two layers there is one weight for every pair of nodes, and every node outside the input layer has one bias. The 4-3-2-1 network has $15 + 8 + 3 = 26$ parameters.

A **trainable parameter** is a number the training algorithm must find: every weight and every bias. Knowing how many there are tells us how big the problem is, so it is the first thing to work out for any architecture.

Every node in one layer connects to every node in the next. So between a layer of 4 nodes and a layer of 3 there are $4 \times 3 = 12$ weights. Each of the 3 receiving nodes is a perceptron with its own bias, which adds 3 more.

Counting layer by layer in Figure 1:

Into layer	Weights	Biases	Total
1	$4 \times 3 = 12$	3	15
2	$3 \times 2 = 6$	2	8
3	$2 \times 1 = 2$	1	3
All	20	6	26

1. **In words:** for each layer after the input, multiply its number of nodes by the previous layer's number of nodes, add its number of nodes (the biases), and add up over all layers.
2. **Formula:** with n_l nodes in layer l and L the output layer,

$$\text{parameters} = \sum_{l=1}^L (n_{l-1} n_l + n_l)$$

3. **Example:** with $n_0 = 4$, $n_1 = 3$, $n_2 = 2$, $n_3 = 1$:

$$(4 \times 3 + 3) + (3 \times 2 + 2) + (2 \times 1 + 1) = 15 + 8 + 3 = 26$$

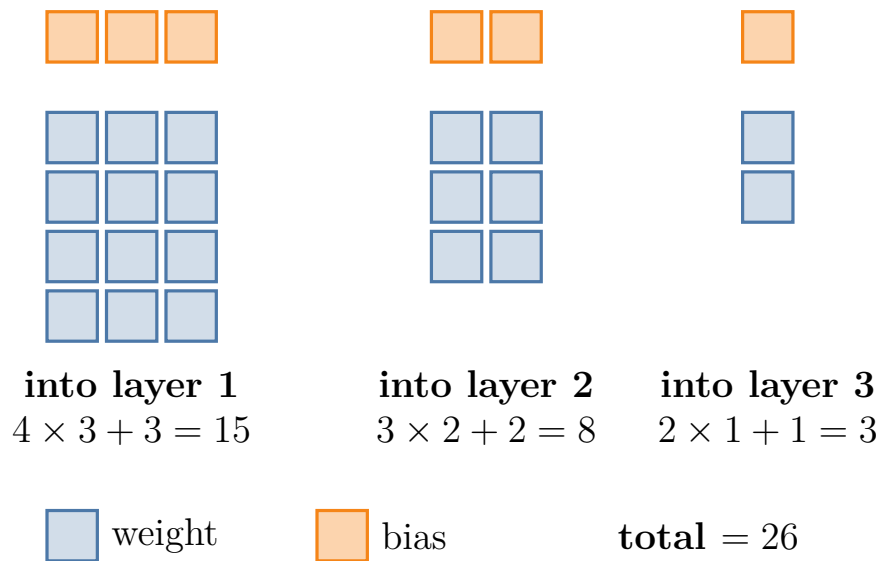


Figure 3: Every parameter of the 4-3-2-1 network as one square. Each block has one weight per pair of nodes (blue) and one bias per receiving node (orange)

Figure 3 is the formula drawn: each block is $n_{l-1} \times n_l$ weights plus a row of n_l biases, and the squares add up to 26.

Training this network means finding good values for these 26 numbers.

Extra: The count grows fast. A network for 28×28 pixel images (784 inputs) with one hidden layer of 128 nodes and 10 outputs already has $784 \times 128 + 128 + 128 \times 10 + 10 = 101,770$ parameters. Keras prints this count for every layer with `model.summary()` (see the [customer churn Note](#)), so it is worth being able to check it by hand.

4. Naming biases and outputs

Key point: b_{ij} is the bias of node j in layer i , and O_{ij} is that node's output. Both use the same two numbers: layer first, then node.

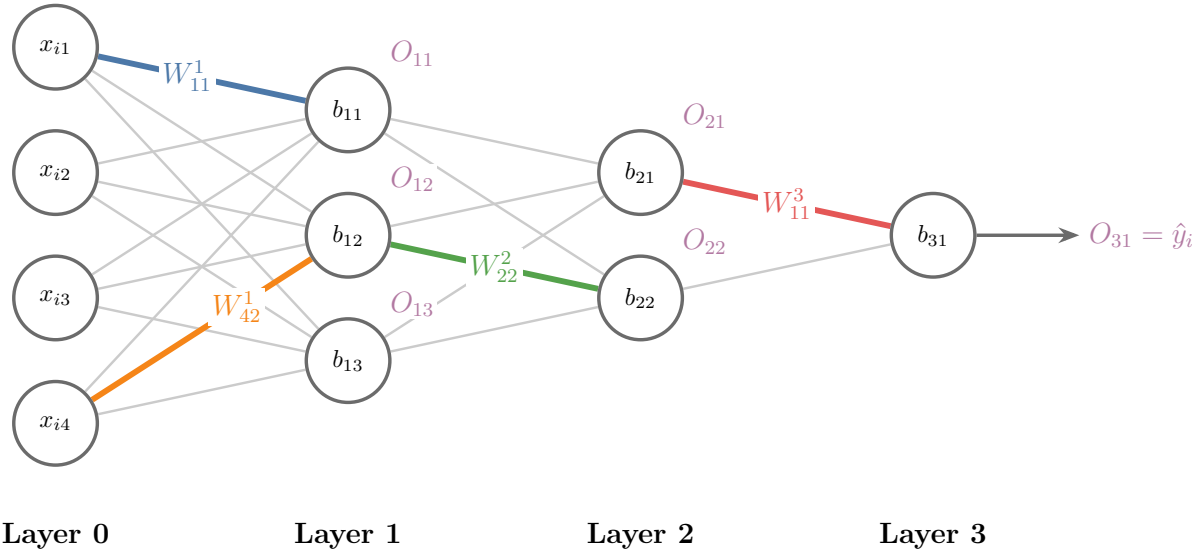


Figure 4: The notation on the same network: a bias inside every node, an output beside it, and four example weights

4.1 Biases

Key point: Layer number, then node number.

The bias of a node is written b_{ij} , where:

- i is the layer number;
- j is the position of the node in that layer, counted from the top.

In Figure 4, the three nodes of layer 1 have biases b_{11} , b_{12} and b_{13} . Layer 2 has b_{21} and b_{22} , and the output node has b_{31} . The input layer has no biases, because its nodes do no calculation.

4.2 Outputs

Key point: The output of node j in layer i is O_{ij} ; it is the input to every node of the next layer.

The output of a node uses exactly the same two indices: O_{ij} . A node's output goes along every connection to the next layer, but it is one number, so all those connections carry the same O_{ij} .

- Node 1 of layer 1 sends O_{11} to both nodes of layer 2.

- Node 2 of layer 2 sends O_{22} to the output node.
- The output node gives O_{31} , which is the prediction $\hat{y}_i$.

5. Naming weights

Key point: W_{ij}^k is the weight going into layer k , leaving node i of the previous layer and entering node j of layer k .

A weight sits on a connection between two nodes, so it needs three numbers:

- k (top): the layer the weight **enters**;
- i (first bottom index): the node it **leaves**, in layer $k - 1$;
- j (second bottom index): the node it **enters**, in layer k .

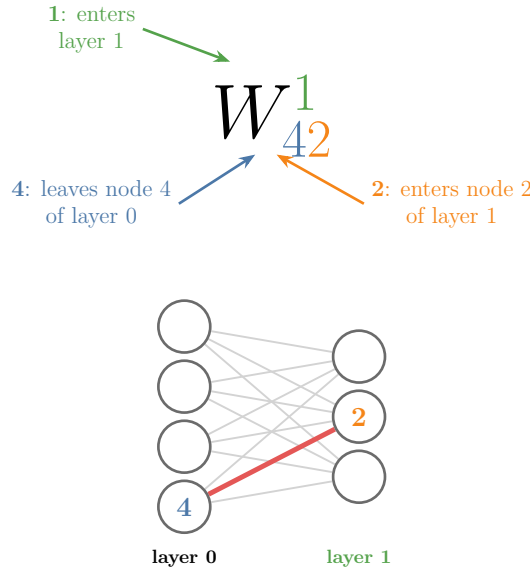


Figure 5: How to read W_{42}^1 : the top index is the layer the weight enters; the bottom indices are the node it leaves and the node it enters

Figure 5 decodes one weight: W_{42}^1 is the red connection from node 4 of layer 0 into node 2 of layer 1.

So W_{ij}^k reads “into layer k , from node i to node j ”. The four highlighted weights in Figure 4:

Weight	Enters layer	Leaves node	Enters node
W_{11}^1	1	1 of layer 0	1 of layer 1
W_{42}^1	1	4 of layer 0	2 of layer 1
W_{22}^2	2	2 of layer 1	2 of layer 2
W_{11}^3	3	1 of layer 2	1 of layer 3

The colours in Figure 1 follow the same idea. All the weights entering one node share that node's colour: the 4 blue weights are $W_{11}^1, W_{21}^1, W_{31}^1, W_{41}^1$, all entering node 1 of layer 1. The blue weights are the ones that node uses in its weighted sum, together with its bias b_{11} .

Extra: Books do not all agree on this order. Some write the weight from node i to node j as w_{ji} (destination first), because then row j of the weight matrix holds the weights entering node j , and a layer's weighted sums are simply $W\mathbf{x}$ (Bishop, §5.1, eq. 5.2). The meaning is the same; only the order of the indices changes. Whichever we use, we keep it fixed, so that a symbol never has two meanings.

Python: The same parameters as NumPy arrays.

```
import numpy as np
```

```
sizes = [4, 3, 2, 1]
```

```
# W[k] has shape (nodes in layer k-1, nodes in layer k)
```

```
# so W[k][i-1, j-1] is the weight W^k_ij
```

```
W = {k: np.zeros((sizes[k-1], sizes[k])) for k in (1, 2, 3)}
```

```
b = {k: np.zeros(sizes[k]) for k in (1, 2, 3)}
```

```
sum(W[k].size + b[k].size for k in (1, 2, 3))    # 26
```

Python counts from 0, so W_{42}^1 is $W[1][3, 1]$. Storing each layer's weights as one matrix is exactly what the [forward propagation Note](#) uses to compute a prediction.

6. Summary

Symbol	Meaning	Example
x_{ij}	Value of feature j for observation i	x_{i3} : the 10th marks of student i
b_{ij}	Bias of node j in layer i	b_{22} : second node of layer 2
O_{ij}	Output of node j in layer i	$O_{31} = \hat{y}_i$
W_{ij}^k	Weight into layer k , from node i to node j	W_{42}^1 : input 4 to node 2 of layer 1

- Layers are numbered from 0 (input) to the output layer.
- Parameters = $\sum_l (n_{l-1}n_l + n_l)$; the 4-3-2-1 network has 26.
- Biases and outputs use (layer, node); weights add the layer they enter on top.

7. Sources

Built from

- CampusX, “MLP Notation”, YouTube, https://www.youtube.com/watch?v=H0_3SJh4Rqs

Other references

- Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, §5.1 (eq. 5.2 writes the weight into hidden unit j from input i as w_{ji}).

8. Key terms

Term	Meaning
Trainable parameter	A weight or bias whose value training must find
Layer number	The position of a layer, from 0 for the input layer to the output layer
4-3-2-1 network	A network described by its layer sizes, input first
b_{ij}	The bias of node j in layer i
O_{ij}	The output of node j in layer i
W_{ij}^k	The weight entering layer k , from node i of layer $k - 1$ to node j of layer k